

main-svm

November 29, 2025

1 SVM

```
[27]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score,
)
```

```
[28]: # Load the cleaned dataset
df = pd.read_csv("heart-cleaned.csv")
print(f"Dataset shape: {df.shape}")
print(f"\nFirst few rows:")
df.head()
```

Dataset shape: (746, 12)

First few rows:

```
[28]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0

1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

```
[29]: # Separate features and target
X = df.drop(columns=["HeartDisease"])
y = df["HeartDisease"]

# Identify numerical and categorical columns
numerical_cols = ["Age", "RestingBP", "Cholesterol", "FastingBS", "MaxHR", "Oldpeak"]
categorical_cols = ["Sex", "ChestPainType", "RestingECG", "ExerciseAngina", "ST_Slope"]

print(f"Numerical columns: {numerical_cols}")
print(f"Categorical columns: {categorical_cols}")
print(f"\nTarget distribution:")
print(y.value_counts())
```

Numerical columns: ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']

Categorical columns: ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']

Target distribution:

HeartDisease

0 390

1 356

Name: count, dtype: int64

```
[30]: # One-hot encode categorical variables
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
print(f"Shape after encoding: {X_encoded.shape}")
print(f"\nFeature names:")
print(X_encoded.columns.tolist())
```

Shape after encoding: (746, 15)

Feature names:

['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'Sex_M', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA', 'RestingECG_Normal', 'RestingECG_ST', 'ExerciseAngina_Y', 'ST_Slope_Flat', 'ST_Slope_Up']

```
[31]: # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y)
```

```
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
print(f"\nTraining target distribution:")
print(y_train.value_counts())
print(f"\nTest target distribution:")
print(y_test.value_counts())
```

Training set size: 596

Test set size: 150

Training target distribution:

HeartDisease

0 312

1 284

Name: count, dtype: int64

Test target distribution:

HeartDisease

0 78

1 72

Name: count, dtype: int64

```
[32]: # Define function for hyperparameter tuning
def tune_clf_hyperparameters(
    pipeline, param_grid, X_train, y_train, cv=5, scoring="accuracy"
):
    grid_search = GridSearchCV(
        pipeline, param_grid, cv=cv, scoring=scoring, n_jobs=-1, verbose=1
    )
    grid_search.fit(X_train, y_train)

    print(f"\nBest cross-validation score: {grid_search.best_score_:.4f}")
    print(f"Best parameters: {grid_search.best_params_}\n")

    return grid_search.best_estimator_, grid_search.best_params_
```

```
[33]: # Create SVM pipeline with standardization
svm_pipeline = Pipeline([("scaler", StandardScaler()), ("svm",
    ↪SVC(random_state=42))])

# Configure hyperparameter grid
param_grid_svm = {
    "svm__C": [0.001, 0.005, 0.01, 0.05, 0.1, 1, 10, 20],
    "svm__kernel": ["linear", "rbf", "poly"],
    "svm__gamma": ["scale", "auto", 0.1, 0.5, 1, 5],
```

```

        "svm__degree": [2, 3, 4],
    }

    # Perform GridSearchCV to get all results
    grid_search = GridSearchCV(
        svm_pipeline, param_grid_svm, cv=5, scoring="accuracy", n_jobs=-1, verbose=1
    )
    grid_search.fit(X_train, y_train)

    # Get best model and parameters
    best_svm = grid_search.best_estimator_
    best_svm_hyperparams = grid_search.best_params_

    print("\n" + "=" * 70)
    print("SVM Optimal Hyperparameters:")
    print("=" * 70)
    for param, value in best_svm_hyperparams.items():
        print(f"    {param}: {value}")
    print(f"\nBest cross-validation score: {grid_search.best_score_:.4f}")
    print("=" * 70)

    # Create DataFrame with all results
    results_df = pd.DataFrame(grid_search.cv_results_)

    # Select relevant columns and sort by mean test score
    results_display = results_df[
        [
            "param_svm__C",
            "param_svm__kernel",
            "param_svm__gamma",
            "param_svm__degree",
            "mean_test_score",
            "std_test_score",
        ]
    ].copy()

    # Sort by mean test score in descending order
    results_display = results_display.sort_values("mean_test_score",
        ↪ascending=False)

    # Rename columns for better readability
    results_display.columns = ["C", "Kernel", "Gamma", "Degree", "Mean CV Score",
        ↪"Std CV Score"]

    print("\n" + "=" * 100)
    print("ALL HYPERPARAMETER COMBINATIONS RESULTS (Sorted by Mean CV Score)")
    print("=" * 100)

```

```

print(f"\nTotal combinations tested: {len(results_display)}")
print(f"\nTop 20 combinations:")
print("-" * 100)
print(results_display.head(20).to_string(index=False))
print("\n" + "-" * 100)

# Display all results (can be scrolled in notebook)
print(f"\n\nALL {len(results_display)} COMBINATIONS (Complete Results):")
print("=" * 100)
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
print(results_display.to_string(index=False))
pd.reset_option('display.max_rows')
pd.reset_option('display.max_columns')
pd.reset_option('display.width')
pd.reset_option('display.max_colwidth')
print("\n" + "=" * 100)

# Group results by kernel to show summary
print("\n" + "=" * 100)
print("SUMMARY BY KERNEL TYPE")
print("=" * 100)
for kernel in ["linear", "rbf", "poly"]:
    kernel_results = results_display[results_display["Kernel"] == kernel]
    if len(kernel_results) > 0:
        print(f"\n{kernel.upper()} Kernel:")
        print(f"  Best score: {kernel_results['Mean CV Score'].max():.4f}")
        print(f"  Worst score: {kernel_results['Mean CV Score'].min():.4f}")
        print(f"  Average score: {kernel_results['Mean CV Score'].mean():.4f}")
        print(f"  Number of combinations: {len(kernel_results)}")
        best_kernel_combo = kernel_results.iloc[0]
        print(f"  Best combination: C={best_kernel_combo['C']},  

↳ Gamma={best_kernel_combo['Gamma']}, Degree={best_kernel_combo['Degree']}")

print("\n" + "=" * 100)

```

Fitting 5 folds for each of 432 candidates, totalling 2160 fits

```

=====
SVM Optimal Hyperparameters:
=====

```

```

svm__C: 0.05
svm__degree: 2
svm__gamma: scale
svm__kernel: rbf

```

Best cross-validation score: 0.8674

=====

=====

ALL HYPERPARAMETER COMBINATIONS RESULTS (Sorted by Mean CV Score)

=====

Total combinations tested: 432

Top 20 combinations:

C	Kernel	Gamma	Degree	Mean CV Score	Std CV Score
0.050	rbf	auto	2	0.867409	0.020383
0.050	rbf	scale	3	0.867409	0.020383
0.050	rbf	auto	3	0.867409	0.020383
0.050	rbf	scale	4	0.867409	0.020383
0.050	rbf	auto	4	0.867409	0.020383
0.050	rbf	scale	2	0.867409	0.020383
0.050	rbf	0.1	2	0.862381	0.024289
0.050	rbf	0.1	3	0.862381	0.024289
0.050	rbf	0.1	4	0.862381	0.024289
0.100	rbf	scale	2	0.862381	0.013946
0.100	rbf	auto	4	0.862381	0.013946
0.100	rbf	auto	2	0.862381	0.013946
0.100	rbf	scale	3	0.862381	0.013946
0.100	rbf	scale	4	0.862381	0.013946
0.100	rbf	auto	3	0.862381	0.013946
0.100	poly	auto	3	0.859048	0.014568
0.100	poly	scale	3	0.859048	0.014568
0.001	poly	0.5	3	0.859034	0.021013
0.001	linear	scale	4	0.857367	0.019985
0.001	linear	5	3	0.857367	0.019985

ALL 432 COMBINATIONS (Complete Results):

=====

C	Kernel	Gamma	Degree	Mean CV Score	Std CV Score
0.050	rbf	auto	2	0.867409	0.020383
0.050	rbf	scale	3	0.867409	0.020383
0.050	rbf	auto	3	0.867409	0.020383
0.050	rbf	scale	4	0.867409	0.020383

0.050	rbf	auto	4	0.867409	0.020383
0.050	rbf	scale	2	0.867409	0.020383
0.050	rbf	0.1	2	0.862381	0.024289
0.050	rbf	0.1	3	0.862381	0.024289
0.050	rbf	0.1	4	0.862381	0.024289
0.100	rbf	scale	2	0.862381	0.013946
0.100	rbf	auto	4	0.862381	0.013946
0.100	rbf	auto	2	0.862381	0.013946
0.100	rbf	scale	3	0.862381	0.013946
0.100	rbf	scale	4	0.862381	0.013946
0.100	rbf	auto	3	0.862381	0.013946
0.100	poly	auto	3	0.859048	0.014568
0.100	poly	scale	3	0.859048	0.014568
0.001	poly	0.5	3	0.859034	0.021013
0.001	linear	scale	4	0.857367	0.019985
0.001	linear	5	3	0.857367	0.019985
0.001	linear	auto	2	0.857367	0.019985
0.001	linear	5	4	0.857367	0.019985
0.001	linear	0.1	2	0.857367	0.019985
0.001	linear	1	4	0.857367	0.019985
0.100	rbf	0.1	4	0.857367	0.015164
0.001	linear	0.5	4	0.857367	0.019985
0.001	linear	0.5	2	0.857367	0.019985
0.001	linear	0.1	4	0.857367	0.019985
0.001	linear	auto	4	0.857367	0.019985
0.100	rbf	0.1	2	0.857367	0.015164
0.001	linear	scale	3	0.857367	0.019985
0.001	linear	scale	2	0.857367	0.019985
0.001	linear	5	2	0.857367	0.019985
0.001	linear	1	3	0.857367	0.019985
0.100	rbf	0.1	3	0.857367	0.015164
0.001	linear	0.5	3	0.857367	0.019985
0.001	linear	auto	3	0.857367	0.019985
0.001	linear	1	2	0.857367	0.019985
0.001	linear	0.1	3	0.857367	0.019985
0.100	linear	0.1	4	0.857353	0.023920
0.100	linear	5	4	0.857353	0.023920
0.100	linear	0.5	3	0.857353	0.023920
0.100	linear	1	4	0.857353	0.023920
0.100	linear	0.5	4	0.857353	0.023920
0.100	linear	0.1	3	0.857353	0.023920
0.100	linear	5	3	0.857353	0.023920
0.100	linear	scale	3	0.857353	0.023920
0.100	linear	auto	2	0.857353	0.023920
0.100	linear	auto	3	0.857353	0.023920
0.100	linear	auto	4	0.857353	0.023920
0.100	linear	0.1	2	0.857353	0.023920
0.100	linear	scale	4	0.857353	0.023920

0.100	linear	0.5	2	0.857353	0.023920
0.100	linear	1	2	0.857353	0.023920
0.100	linear	5	2	0.857353	0.023920
0.100	linear	1	3	0.857353	0.023920
0.100	linear	scale	2	0.857353	0.023920
20.000	linear	0.5	3	0.857339	0.025129
1.000	linear	1	4	0.857339	0.025129
20.000	linear	5	3	0.857339	0.025129
10.000	linear	scale	2	0.857339	0.025129
10.000	linear	auto	2	0.857339	0.025129
20.000	linear	1	3	0.857339	0.025129
10.000	linear	0.1	2	0.857339	0.025129
10.000	linear	1	4	0.857339	0.025129
10.000	linear	0.5	2	0.857339	0.025129
10.000	linear	1	2	0.857339	0.025129
10.000	linear	5	2	0.857339	0.025129
10.000	linear	scale	3	0.857339	0.025129
10.000	linear	auto	3	0.857339	0.025129
10.000	linear	0.1	3	0.857339	0.025129
20.000	linear	0.1	3	0.857339	0.025129
10.000	linear	0.5	3	0.857339	0.025129
10.000	linear	0.5	4	0.857339	0.025129
20.000	linear	auto	3	0.857339	0.025129
20.000	linear	scale	3	0.857339	0.025129
10.000	linear	1	3	0.857339	0.025129
10.000	linear	5	4	0.857339	0.025129
20.000	linear	5	2	0.857339	0.025129
10.000	linear	5	3	0.857339	0.025129
10.000	linear	0.1	4	0.857339	0.025129
20.000	linear	1	2	0.857339	0.025129
20.000	linear	scale	2	0.857339	0.025129
20.000	linear	0.5	2	0.857339	0.025129
10.000	linear	scale	4	0.857339	0.025129
20.000	linear	0.1	2	0.857339	0.025129
20.000	linear	auto	2	0.857339	0.025129
1.000	linear	0.5	3	0.857339	0.025129
1.000	linear	5	4	0.857339	0.025129
10.000	linear	auto	4	0.857339	0.025129
20.000	linear	scale	4	0.857339	0.025129
1.000	linear	1	3	0.857339	0.025129
1.000	linear	1	2	0.857339	0.025129
20.000	linear	0.5	4	0.857339	0.025129
1.000	linear	0.5	2	0.857339	0.025129
1.000	linear	auto	3	0.857339	0.025129
1.000	linear	0.1	2	0.857339	0.025129
1.000	linear	auto	2	0.857339	0.025129
20.000	linear	1	4	0.857339	0.025129
1.000	linear	scale	3	0.857339	0.025129

20.000	linear	0.1	4	0.857339	0.025129
1.000	linear	scale	2	0.857339	0.025129
1.000	linear	5	2	0.857339	0.025129
1.000	linear	5	3	0.857339	0.025129
1.000	linear	auto	4	0.857339	0.025129
1.000	linear	0.5	4	0.857339	0.025129
1.000	linear	0.1	4	0.857339	0.025129
20.000	linear	5	4	0.857339	0.025129
1.000	linear	0.1	3	0.857339	0.025129
1.000	linear	scale	4	0.857339	0.025129
20.000	linear	auto	4	0.857339	0.025129
0.050	poly	0.1	3	0.855686	0.017277
1.000	poly	scale	3	0.855686	0.022901
1.000	poly	auto	3	0.855686	0.022901
0.100	poly	0.1	3	0.855672	0.018926
0.050	linear	0.1	3	0.852311	0.021213
0.050	linear	scale	3	0.852311	0.021213
0.050	linear	0.5	2	0.852311	0.021213
0.050	linear	0.5	3	0.852311	0.021213
0.050	linear	0.1	2	0.852311	0.021213
0.050	linear	1	3	0.852311	0.021213
0.050	linear	auto	3	0.852311	0.021213
0.050	linear	auto	2	0.852311	0.021213
0.050	linear	0.5	4	0.852311	0.021213
0.050	linear	5	3	0.852311	0.021213
0.050	linear	1	2	0.852311	0.021213
0.050	linear	5	4	0.852311	0.021213
0.050	linear	1	4	0.852311	0.021213
0.050	linear	scale	2	0.852311	0.021213
0.050	linear	5	2	0.852311	0.021213
0.050	linear	0.1	4	0.852311	0.021213
0.050	linear	auto	4	0.852311	0.021213
0.050	linear	scale	4	0.852311	0.021213
0.005	poly	0.5	3	0.852311	0.017572
1.000	rbf	auto	2	0.850658	0.009973
1.000	rbf	auto	3	0.850658	0.009973
1.000	rbf	scale	2	0.850658	0.009973
1.000	rbf	scale	3	0.850658	0.009973
1.000	rbf	scale	4	0.850658	0.009973
1.000	rbf	auto	4	0.850658	0.009973
0.010	linear	0.1	4	0.848992	0.016814
0.010	linear	1	4	0.848992	0.016814
0.010	linear	auto	4	0.848992	0.016814
0.010	linear	0.5	4	0.848992	0.016814
0.010	linear	scale	2	0.848992	0.016814
0.010	linear	scale	4	0.848992	0.016814
0.010	linear	5	3	0.848992	0.016814
0.010	linear	1	3	0.848992	0.016814

0.010	linear	0.5	3	0.848992	0.016814
0.010	linear	0.1	3	0.848992	0.016814
0.010	linear	auto	3	0.848992	0.016814
0.010	linear	scale	3	0.848992	0.016814
0.010	linear	5	2	0.848992	0.016814
0.010	linear	1	2	0.848992	0.016814
0.010	linear	0.5	2	0.848992	0.016814
0.010	linear	0.1	2	0.848992	0.016814
0.010	linear	auto	2	0.848992	0.016814
0.010	linear	5	4	0.848992	0.016814
0.010	poly	0.5	3	0.848964	0.017845
0.001	poly	1	3	0.847283	0.015677
1.000	poly	0.1	3	0.847283	0.015677
0.050	poly	auto	3	0.845630	0.019649
0.050	poly	scale	3	0.845630	0.019649
1.000	rbf	0.1	2	0.845588	0.026590
1.000	rbf	0.1	3	0.845588	0.026590
1.000	rbf	0.1	4	0.845588	0.026590
0.005	linear	auto	2	0.843964	0.019561
0.005	linear	0.1	4	0.843964	0.019561
0.005	linear	1	4	0.843964	0.019561
0.005	linear	5	4	0.843964	0.019561
0.005	linear	auto	4	0.843964	0.019561
0.005	linear	scale	4	0.843964	0.019561
0.005	linear	5	3	0.843964	0.019561
0.005	linear	0.5	3	0.843964	0.019561
0.005	linear	0.1	3	0.843964	0.019561
0.005	linear	auto	3	0.843964	0.019561
0.005	linear	scale	3	0.843964	0.019561
0.005	linear	1	3	0.843964	0.019561
0.005	linear	0.5	4	0.843964	0.019561
0.005	linear	scale	2	0.843964	0.019561
0.005	linear	1	2	0.843964	0.019561
0.005	linear	0.1	2	0.843964	0.019561
0.005	linear	0.5	2	0.843964	0.019561
0.005	linear	5	2	0.843964	0.019561
10.000	poly	auto	3	0.835532	0.011856
10.000	poly	scale	3	0.835532	0.011856
0.100	poly	0.1	4	0.825490	0.016419
10.000	rbf	auto	3	0.825462	0.022441
10.000	rbf	auto	2	0.825462	0.022441
10.000	rbf	auto	4	0.825462	0.022441
10.000	rbf	scale	2	0.825462	0.022441
10.000	rbf	scale	3	0.825462	0.022441
10.000	rbf	scale	4	0.825462	0.022441
10.000	rbf	0.1	4	0.818768	0.024284
10.000	rbf	0.1	2	0.818768	0.024284
10.000	rbf	0.1	3	0.818768	0.024284

1.000	poly	scale	4	0.815420	0.026673
1.000	poly	auto	4	0.815420	0.026673
1.000	rbf	0.5	4	0.812017	0.032878
1.000	rbf	0.5	3	0.812017	0.032878
1.000	rbf	0.5	2	0.812017	0.032878
20.000	rbf	scale	2	0.808697	0.027986
20.000	rbf	auto	3	0.808697	0.027986
20.000	rbf	scale	3	0.808697	0.027986
20.000	rbf	scale	4	0.808697	0.027986
20.000	rbf	auto	2	0.808697	0.027986
20.000	rbf	auto	4	0.808697	0.027986
20.000	rbf	0.1	2	0.805336	0.026994
20.000	rbf	0.1	3	0.805336	0.026994
20.000	rbf	0.1	4	0.805336	0.026994
10.000	poly	auto	2	0.803599	0.036248
10.000	poly	scale	2	0.803599	0.036248
0.010	poly	1	3	0.801961	0.028725
10.000	poly	0.1	3	0.801961	0.028725
0.050	poly	0.5	3	0.801947	0.023372
0.005	poly	5	2	0.801933	0.033381
0.001	poly	0.5	4	0.800308	0.027994
0.005	poly	1	3	0.800266	0.026129
20.000	poly	auto	3	0.800266	0.025025
20.000	poly	scale	3	0.800266	0.025025
0.050	poly	1	2	0.798571	0.033575
0.100	poly	0.5	3	0.796947	0.034777
10.000	rbf	0.5	2	0.796933	0.030977
20.000	rbf	0.5	4	0.796933	0.030977
10.000	rbf	0.5	4	0.796933	0.030977
20.000	rbf	0.5	3	0.796933	0.030977
20.000	rbf	0.5	2	0.796933	0.030977
10.000	rbf	0.5	3	0.796933	0.030977
0.050	poly	5	2	0.796933	0.036811
0.100	poly	5	2	0.796919	0.039095
10.000	poly	0.5	2	0.796919	0.039095
0.010	poly	0.1	3	0.795308	0.020919
1.000	poly	0.1	4	0.795280	0.030010
20.000	poly	0.5	2	0.793585	0.036471
10.000	poly	0.1	2	0.793571	0.030643
0.100	poly	1	2	0.793571	0.030643
20.000	poly	0.1	2	0.791905	0.040450
20.000	poly	scale	2	0.791891	0.032362
20.000	poly	auto	2	0.791891	0.032362
10.000	poly	1	2	0.791877	0.039491
10.000	poly	auto	4	0.790238	0.026784
10.000	poly	scale	4	0.790238	0.026784
0.001	poly	5	2	0.790182	0.035238
0.100	poly	0.5	2	0.790182	0.035238

20.000	poly	0.1	3	0.788557	0.023066
1.000	poly	0.1	2	0.786849	0.036584
0.010	poly	1	2	0.786849	0.036584
0.050	poly	0.5	2	0.786849	0.027299
1.000	poly	0.5	2	0.786835	0.044628
0.010	poly	5	2	0.786835	0.044628
1.000	poly	1	2	0.785196	0.034078
0.050	poly	1	3	0.783529	0.021768
0.010	poly	0.5	2	0.781863	0.018583
0.005	poly	0.5	2	0.781849	0.025718
1.000	poly	5	2	0.780140	0.039122
20.000	poly	1	2	0.780140	0.036118
0.001	poly	1	2	0.778501	0.037547
0.100	poly	0.1	2	0.778501	0.037547
0.005	poly	1	2	0.778473	0.033304
0.005	poly	0.5	4	0.776807	0.027171
20.000	poly	5	2	0.773417	0.046312
0.001	poly	5	3	0.771863	0.035828
1.000	poly	0.5	3	0.771863	0.035828
1.000	poly	auto	2	0.770084	0.029253
1.000	poly	scale	2	0.770084	0.029253
20.000	poly	scale	4	0.770070	0.037377
20.000	poly	auto	4	0.770070	0.037377
0.100	poly	1	3	0.768487	0.033624
10.000	poly	5	2	0.768347	0.049538
10.000	poly	0.5	3	0.766793	0.026645
20.000	poly	0.5	3	0.766793	0.026645
0.005	poly	5	3	0.766793	0.026645
0.050	poly	5	3	0.766793	0.026645
10.000	poly	1	3	0.766793	0.026645
10.000	poly	5	3	0.766793	0.026645
20.000	poly	1	3	0.766793	0.026645
0.010	poly	5	3	0.766793	0.026645
1.000	poly	5	3	0.766793	0.026645
20.000	poly	5	3	0.766793	0.026645
0.100	poly	5	3	0.766793	0.026645
1.000	poly	1	3	0.766793	0.026645
0.050	poly	0.1	4	0.766737	0.037581
0.010	poly	0.5	4	0.758319	0.038095
0.005	poly	1	4	0.756681	0.040986
0.050	poly	0.5	4	0.753291	0.042363
0.100	poly	0.5	4	0.749972	0.038267
10.000	poly	0.1	4	0.749930	0.039206
0.001	poly	1	4	0.749930	0.039206
20.000	poly	0.1	4	0.748277	0.030851
10.000	poly	1	4	0.746611	0.035228
20.000	poly	1	4	0.746611	0.035228
1.000	poly	0.5	4	0.746611	0.035228

20.000	poly	0.5	4	0.746611	0.035228
10.000	poly	5	4	0.746611	0.035228
10.000	poly	0.5	4	0.746611	0.035228
1.000	poly	1	4	0.746611	0.035228
1.000	poly	5	4	0.746611	0.035228
20.000	poly	5	4	0.746611	0.035228
0.001	poly	5	4	0.746611	0.035228
0.100	poly	5	4	0.746611	0.035228
0.010	poly	5	4	0.746611	0.035228
0.050	poly	5	4	0.746611	0.035228
0.100	poly	1	4	0.746611	0.035228
0.050	poly	1	4	0.746611	0.035228
0.010	poly	1	4	0.746611	0.035228
0.005	poly	5	4	0.746611	0.035228
1.000	rbf	1	3	0.738179	0.042032
1.000	rbf	1	4	0.738179	0.042032
1.000	rbf	1	2	0.738179	0.042032
20.000	rbf	1	3	0.736499	0.040051
10.000	rbf	1	2	0.736499	0.040051
10.000	rbf	1	4	0.736499	0.040051
20.000	rbf	1	2	0.736499	0.040051
10.000	rbf	1	3	0.736499	0.040051
20.000	rbf	1	4	0.736499	0.040051
0.050	poly	0.1	2	0.677871	0.038733
0.100	poly	scale	2	0.650994	0.037525
0.100	poly	auto	2	0.650994	0.037525
0.100	poly	scale	4	0.602339	0.029022
0.100	poly	auto	4	0.602339	0.029022
0.005	poly	0.1	3	0.592255	0.009637
0.050	poly	scale	4	0.563739	0.015493
0.050	poly	auto	4	0.563739	0.015493
0.010	poly	0.1	4	0.562059	0.014044
0.001	poly	0.5	2	0.562031	0.020428
10.000	rbf	5	4	0.557031	0.016656
10.000	rbf	5	3	0.557031	0.016656
20.000	rbf	5	2	0.557031	0.016656
20.000	rbf	5	3	0.557031	0.016656
20.000	rbf	5	4	0.557031	0.016656
10.000	rbf	5	2	0.557031	0.016656
1.000	rbf	5	2	0.552017	0.014468
1.000	rbf	5	4	0.552017	0.014468
1.000	rbf	5	3	0.552017	0.014468
0.100	rbf	0.5	4	0.543627	0.006309
0.100	rbf	0.5	3	0.543627	0.006309
0.100	rbf	0.5	2	0.543627	0.006309
0.050	poly	auto	2	0.541975	0.020273
0.050	poly	scale	2	0.541975	0.020273
0.005	poly	0.1	4	0.535252	0.010536

0.010	poly	auto	3	0.528529	0.009373
0.010	poly	scale	3	0.528529	0.009373
0.010	rbf	0.1	2	0.523487	0.003341
0.005	rbf	1	4	0.523487	0.003341
0.005	rbf	5	4	0.523487	0.003341
0.010	rbf	scale	2	0.523487	0.003341
0.010	poly	scale	2	0.523487	0.003341
0.010	rbf	auto	2	0.523487	0.003341
0.010	poly	auto	2	0.523487	0.003341
0.010	poly	0.1	2	0.523487	0.003341
0.005	rbf	0.1	2	0.523487	0.003341
0.010	rbf	0.5	2	0.523487	0.003341
0.001	poly	auto	2	0.523487	0.003341
0.010	rbf	1	2	0.523487	0.003341
0.010	rbf	5	2	0.523487	0.003341
0.001	rbf	0.1	2	0.523487	0.003341
0.010	rbf	scale	3	0.523487	0.003341
0.010	rbf	auto	3	0.523487	0.003341
0.010	rbf	0.1	3	0.523487	0.003341
0.005	rbf	0.5	4	0.523487	0.003341
0.005	poly	auto	2	0.523487	0.003341
0.005	rbf	0.1	4	0.523487	0.003341
0.005	rbf	0.1	3	0.523487	0.003341
0.001	poly	scale	2	0.523487	0.003341
0.005	poly	0.1	2	0.523487	0.003341
0.005	rbf	0.5	2	0.523487	0.003341
0.005	rbf	1	2	0.523487	0.003341
0.005	rbf	5	2	0.523487	0.003341
0.005	rbf	scale	3	0.523487	0.003341
0.005	poly	scale	3	0.523487	0.003341
0.005	rbf	auto	3	0.523487	0.003341
0.005	poly	auto	3	0.523487	0.003341
0.100	rbf	5	4	0.523487	0.003341
0.010	rbf	1	3	0.523487	0.003341
0.005	rbf	0.5	3	0.523487	0.003341
0.005	rbf	1	3	0.523487	0.003341
0.005	rbf	5	3	0.523487	0.003341
0.005	rbf	scale	4	0.523487	0.003341
0.100	rbf	1	4	0.523487	0.003341
0.005	poly	scale	4	0.523487	0.003341
0.005	rbf	auto	4	0.523487	0.003341
0.001	rbf	auto	2	0.523487	0.003341
0.005	poly	auto	4	0.523487	0.003341
0.010	rbf	0.5	3	0.523487	0.003341
0.001	rbf	5	3	0.523487	0.003341
0.010	rbf	5	3	0.523487	0.003341
0.001	rbf	scale	2	0.523487	0.003341
0.001	rbf	1	4	0.523487	0.003341

0.001	poly	scale	3	0.523487	0.003341
0.001	rbf	auto	3	0.523487	0.003341
0.001	rbf	0.5	4	0.523487	0.003341
0.050	rbf	0.5	4	0.523487	0.003341
0.050	rbf	1	4	0.523487	0.003341
0.001	poly	auto	3	0.523487	0.003341
0.050	rbf	5	4	0.523487	0.003341
0.001	rbf	0.1	3	0.523487	0.003341
0.001	poly	0.1	4	0.523487	0.003341
0.050	rbf	5	3	0.523487	0.003341
0.001	poly	0.1	3	0.523487	0.003341
0.001	rbf	0.1	4	0.523487	0.003341
0.001	poly	auto	4	0.523487	0.003341
0.001	rbf	0.5	3	0.523487	0.003341
0.001	rbf	1	3	0.523487	0.003341
0.001	rbf	auto	4	0.523487	0.003341
0.100	rbf	5	2	0.523487	0.003341
0.001	poly	scale	4	0.523487	0.003341
0.001	rbf	scale	4	0.523487	0.003341
0.001	rbf	scale	3	0.523487	0.003341
0.050	rbf	1	3	0.523487	0.003341
0.100	rbf	1	2	0.523487	0.003341
0.100	rbf	5	3	0.523487	0.003341
0.010	poly	scale	4	0.523487	0.003341
0.010	rbf	auto	4	0.523487	0.003341
0.010	poly	auto	4	0.523487	0.003341
0.010	rbf	0.1	4	0.523487	0.003341
0.001	poly	0.1	2	0.523487	0.003341
0.001	rbf	0.5	2	0.523487	0.003341
0.010	rbf	0.5	4	0.523487	0.003341
0.010	rbf	1	4	0.523487	0.003341
0.001	rbf	1	2	0.523487	0.003341
0.010	rbf	5	4	0.523487	0.003341
0.050	rbf	0.5	3	0.523487	0.003341
0.001	rbf	5	2	0.523487	0.003341
0.005	poly	scale	2	0.523487	0.003341
0.005	rbf	scale	2	0.523487	0.003341
0.100	rbf	1	3	0.523487	0.003341
0.050	rbf	0.5	2	0.523487	0.003341
0.050	rbf	1	2	0.523487	0.003341
0.050	rbf	5	2	0.523487	0.003341
0.005	rbf	auto	2	0.523487	0.003341
0.001	rbf	5	4	0.523487	0.003341
0.010	rbf	scale	4	0.523487	0.003341

=====

=====

```
=====
SUMMARY BY KERNEL TYPE
=====
```

LINEAR Kernel:

```
Best score: 0.8574
Worst score: 0.8440
Average score: 0.8540
Number of combinations: 144
Best combination: C=0.001, Gamma=scale, Degree=4
```

RBF Kernel:

```
Best score: 0.8674
Worst score: 0.5235
Average score: 0.6560
Number of combinations: 144
Best combination: C=0.05, Gamma=auto, Degree=2
```

POLY Kernel:

```
Best score: 0.8590
Worst score: 0.5235
Average score: 0.7264
Number of combinations: 144
Best combination: C=0.1, Gamma=auto, Degree=3
```

```
[34]: # Use the best model from hyperparameter tuning
svm_model = best_svm

print("Best SVM model (from hyperparameter tuning) is ready!")
print(
    f"Number of support vectors: {len(svm_model.named_steps['svm'].
    ↪support_vectors_)}"
)
print(f"Support vectors per class: {svm_model.named_steps['svm'].n_support_}")
```

```
Best SVM model (from hyperparameter tuning) is ready!
Number of support vectors: 437
Support vectors per class: [221 216]
```

```
[35]: # Make predictions using the pipeline (it will handle standardization
    ↪automatically)
y_train_pred = svm_model.predict(X_train)
y_test_pred = svm_model.predict(X_test)
```

```

# Calculate accuracies
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("=" * 50)
print("SVM MODEL PERFORMANCE")
print("=" * 50)
print(f"\nTrain Accuracy: {train_accuracy:.4f} ({train_accuracy*100:.2f}%)")
print(f"Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")
print("\n" + "=" * 50)
print("\nConfusion Matrix (Test Set):")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test Set):")
print(
    classification_report(
        y_test, y_test_pred, target_names=["0: No Heart Disease", "1: Heart_
↪Disease"]
    )
)

```

```

=====
SVM MODEL PERFORMANCE
=====

```

```

Train Accuracy: 0.8758 (87.58%)
Test Accuracy: 0.8533 (85.33%)

```

```

=====

```

```

Confusion Matrix (Test Set):
[[67 11]
 [11 61]]

```

```

Classification Report (Test Set):

```

	precision	recall	f1-score	support
0: No Heart Disease	0.86	0.86	0.86	78
1: Heart Disease	0.85	0.85	0.85	72
accuracy			0.85	150
macro avg	0.85	0.85	0.85	150
weighted avg	0.85	0.85	0.85	150

```

[36]: # Plot ROC Curve
print("=" * 70)

```

```

print("ROC CURVE ANALYSIS")
print("=" * 70)

# Get decision function scores (SVM doesn't output probabilities by default)
# We need to use decision_function() to get scores for ROC curve
y_train_scores = svm_model.decision_function(X_train)
y_test_scores = svm_model.decision_function(X_test)

# Calculate ROC curve for test set
fpr, tpr, thresholds = roc_curve(y_test, y_test_scores)
roc_auc = roc_auc_score(y_test, y_test_scores)

# Also calculate for training set for comparison
fpr_train, tpr_train, thresholds_train = roc_curve(y_train, y_train_scores)
roc_auc_train = roc_auc_score(y_train, y_train_scores)

# Plot ROC curves
plt.figure(figsize=(10, 8))
plt.plot(
    fpr, tpr, color="darkorange", lw=2, label=f"Test ROC curve (AUC = {roc_auc:.4f})"
)
plt.plot(
    fpr_train,
    tpr_train,
    color="blue",
    lw=2,
    linestyle="--",
    label=f"Train ROC curve (AUC = {roc_auc_train:.4f})",
)
plt.plot(
    [0, 1],
    [0, 1],
    color="navy",
    lw=2,
    linestyle="--",
    label="Random Classifier (AUC = 0.5000)",
)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate (1 - Specificity)", fontsize=12)
plt.ylabel("True Positive Rate (Sensitivity)", fontsize=12)
plt.title("ROC Curve - SVM Model", fontsize=14, fontweight="bold")
plt.legend(loc="lower right", fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

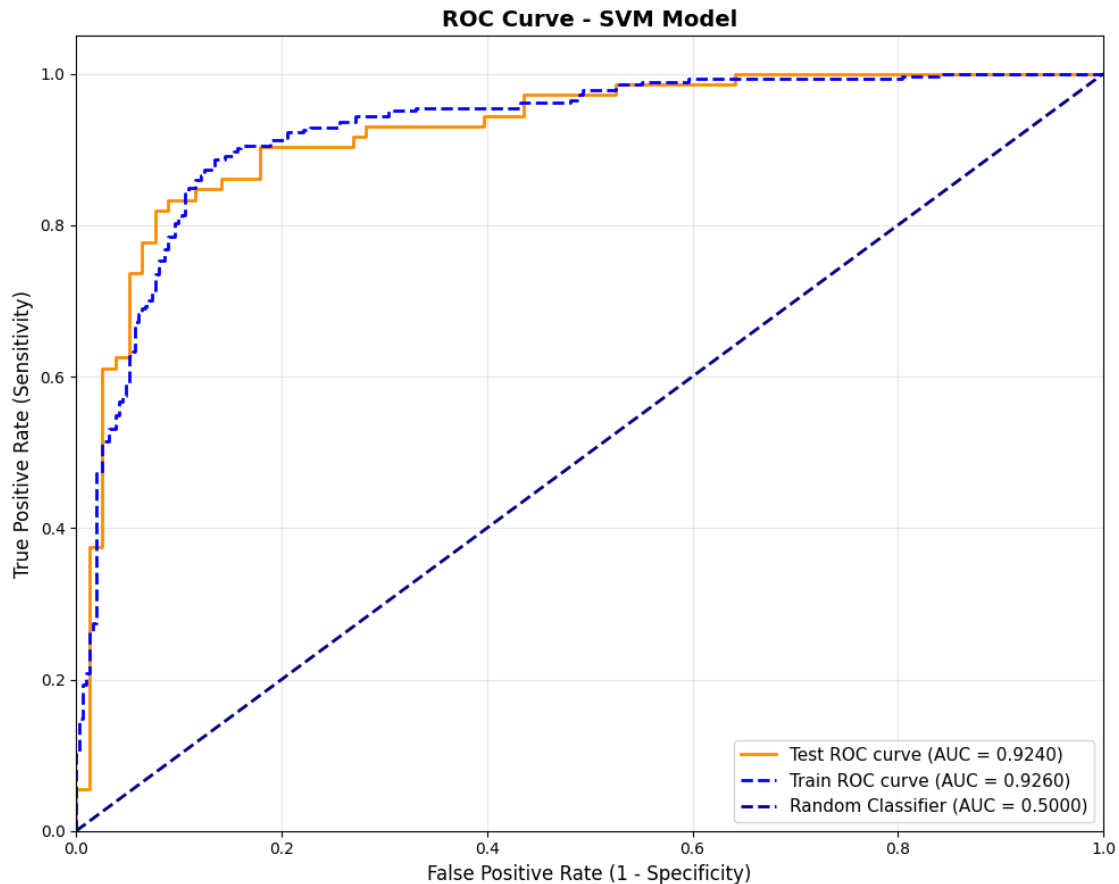
```

```

print(f"\nROC-AUC Score (Test Set): {roc_auc:.4f}")
print(f"ROC-AUC Score (Train Set): {roc_auc_train:.4f}")
print(f"\nInterpretation:")
print(f" - AUC = 0.5: Random classifier (no discriminative ability)")
print(f" - AUC = 1.0: Perfect classifier")
print(f" - AUC > 0.8: Good classifier")
print(f" - AUC > 0.9: Excellent classifier")
print(
    f"\nOur model has an AUC of {roc_auc:.4f}, which indicates {'excellent' if
    ↪roc_auc > 0.9 else 'good' if roc_auc > 0.8 else 'moderate'} performance."
)
print("=" * 70)

```

ROC CURVE ANALYSIS



ROC-AUC Score (Test Set): 0.9240

ROC-AUC Score (Train Set): 0.9260

Interpretation:

- AUC = 0.5: Random classifier (no discriminative ability)
- AUC = 1.0: Perfect classifier
- AUC > 0.8: Good classifier
- AUC > 0.9: Excellent classifier

Our model has an AUC of 0.9240, which indicates excellent performance.

=====

```
[37]: # Plot Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)

group_names = ["True Negative", "False Positive", "False Negative", "True_
↵Positive"]
group_counts = ["{0:0.0f}".format(value) for value in cm.flatten()]
labels = [f"{v1}\n{v2}" for v1, v2 in zip(group_names, group_counts)]
labels = np.asarray(labels).reshape(2, 2)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Negative", "Actual Positive"],
    columns=["Predicted Negative", "Predicted Positive"],
)

plt.figure(figsize=(6, 5))
sns.heatmap(cm_df, annot=labels, fmt="", cmap="Blues", cbar=False)

plt.title("Confusion Matrix - Test Set (SVM)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.show()
```

